

# Case Study 03 — Mixed Theory + Experiment

TD-DFT and measured spectra in one manifest

## Contents

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>1 Case Study 03 — Mixed Theory + Experiment</b>            | <b>1</b> |
| 1.1 What you will produce . . . . .                           | 1        |
| 1.2 1. Prerequisites . . . . .                                | 1        |
| 1.3 2. Inputs . . . . .                                       | 1        |
| 1.4 3. Run with the CLI . . . . .                             | 2        |
| 1.5 4. Run with the HTTP API . . . . .                        | 2        |
| 1.6 5. Drive both interfaces from one Python script . . . . . | 2        |
| 1.7 6. Expected output layout . . . . .                       | 2        |
| 1.8 7. Related case studies . . . . .                         | 3        |

## 1 Case Study 03 — Mixed Theory + Experiment

Build one manifest that contains both Gaussian TD-DFT outputs and experimental Excel spectra, so computed and measured absorption results land in the same `results.csv` under the same broadening, wavelength grid, and Beer–Lambert reference. Each row is tagged with `source_type`  $\in \{\text{"theoretical"}, \text{"experimental"}\}$ .

**Citation.** The bundled experimental data is derived from M. Taniguchi & J. S. Lindsey, *Photochem. Photobiol.* **94** (2018) 290–327. See the project [README](#) for the full citation.

---

### 1.1 What you will produce

Everything is written under one `output/` directory inside this case study folder:

| Folder                                       | By  | Contents                                                                       |
|----------------------------------------------|-----|--------------------------------------------------------------------------------|
| <code>output/cli/tables/</code>              | CLI | Result tables (CSV / JSON / XLSX)                                              |
| <code>output/cli/plots/</code>               | CLI | TIFF plots per sample, light source, and (sample, light) pair                  |
| <code>output/api/</code>                     | API | Same <code>tables/</code> and <code>plots/</code> tree, extracted from the ZIP |
| <code>output/api/analysis_outputs.zip</code> | API | Raw ZIP response                                                               |

Light sources used: **AM1.5G** + **LED-B4**.

---

### 1.2 1. Prerequisites

If you have never run any case study before, follow [Case Study 01 §1 Prerequisites](#) once to install the package, the `requests` library, and either the Python or Docker option for the HTTP API. Every case

study uses the same environment.

### 1.3 2. Inputs

The case-study `input.json` lists 2 TD-DFT outputs and 2 absorbance series from 1 Excel workbook. The manifest uses relative paths that point at the bundled data under `../../input/files/`, so no files are copied or duplicated. To swap in your own data, replace `input.json` with a manifest of the same shape; you do not need to touch `submit.py`.

### 1.4 3. Run with the CLI

Open a terminal at the repository root, then run a single command:

```
overlap-calculator analyze --input case_studies/03_mixed_theory_experiment/input.json --out
→ case_studies/03_mixed_theory_experiment/output/cli --default-light-sources AM15G,LEDB4 --plot-dpi 400
```

After the run, `case_studies/03_mixed_theory_experiment/output/cli/` contains a `tables/` directory (and a `plots/` directory unless plots were disabled). The `--default-light-sources` flag pins the bundled sources to AM1.5G + LED-B4; remove it to compare against the full default set (AM15G,LEDB4,LEDB2,LEDB3,CIEFL10).

### 1.5 4. Run with the HTTP API

Start the API as documented in [Case Study 01 §1](#) and verify with `curl http://localhost:8000/health` (or `:8080` if you started the API via `docker compose`). Then submit the same uploads in one `curl` command:

```
curl -X POST http://localhost:8000/analyze -F "files=@input/files/slurm-5473089.out" -F
→ "files=@input/files/slurm-5473178.out" -F
→ "files=@input/files/organic_uvvis_photochemcad_dataset.xlsx" -F "default_light_sources=AM15G,LEDB4"
→ -F "plot_dpi=400" --output case_studies/03_mixed_theory_experiment/output/api/analysis_outputs.zip
```

Each `-F "files=@..."` is one upload. The form fields mirror the CLI flags. The response is a ZIP archive saved as `output/api/analysis_outputs.zip`.

### 1.6 5. Drive both interfaces from one Python script

Instead of typing the commands above, run `submit.py` from the repository root:

```
python case_studies/03_mixed_theory_experiment/submit.py
```

The script reads `input.json`, runs the CLI step (writing to `output/cli/`), then POSTs the same uploads to `/analyze` and unpacks the response into `output/api/`. If the API is not reachable, it prints a hint and exits cleanly without failing the CLI step.

### 1.7 6. Expected output layout

```
case_studies/03_mixed_theory_experiment/output/
+-- cli/
|   +-- tables/results.{csv,json,xlsx}          (4 samples (2 theoretical + 2 experimental) x 2 light
→   sources = 8 rows)
|   +-- tables/results_timings.{csv,json,xlsx}
|   +-- tables/descriptor_summary.{csv,xlsx}
|   +-- tables/ranking_by_light_source__<metric>.{csv,json,xlsx}  (gaussian/lorentzian x
→   absorbed_fraction/shape_overlap)
|   +-- tables/ranking_by_sample__<metric>.{csv,json,xlsx}        (same four metric variants)
```

```
|  +-- plots/                                     (also  
↪  plots/ranking/by_light_source/<metric>/ and plots/ranking/by_sample/<metric>/)  
+-- api/  
    +-- analysis_outputs.zip  
    +-- tables/...  
    +-- plots/...
```

API sample IDs are auto-generated from each upload's %chk header, while the CLI uses the `sample_id` you set in `input.json`. This is the only cosmetic difference between the two trees; the tables align row by row otherwise.

---

## 1.8 7. Related case studies

- [Case Study 01 — Theoretical Only](#)
- [Case Study 02 — Experimental Only](#)
- [Case Study 11 — Sheet Overrides](#)